

ADR-001 · Arquitectura de la aplicación GameOn Network

- Estado: aceptada
- Fecha: 2026-09-04
- Decidido por: GameOn Team

Contexto

GameOn Network es una aplicación móvil Flutter que consume servicios remotos de Supabase y Firebase e integra geolocalización, Google Maps, notificaciones, reservas, perfiles y chat. El proyecto ya contaba con una estructura funcional basada en models, viewmodels, services y views; sin embargo, el HomeViewModel consultaba directamente a Supabase. Esa dependencia hacía difícil probar la lógica de presentación sin infraestructura real.

El equipo está conformado por tres integrantes y el horizonte académico es de cinco sprints. Se necesita una arquitectura que permita crecer sin introducir una cantidad desproporcionada de capas.

Alternativas consideradas

Alternativa	Ventajas	Desventajas	Adecuación al proyecto
MVC clásico	Simple y conocido	Facilita controladores con demasiadas responsabilidades y acoplamiento a datos	Insuficiente
MVVM + Repository + dominio ligero	ViewModels testeables, separación clara, proporcional al tamaño del equipo	Genera más archivos y exige disciplina	Adecuada
Clean Architecture completa	Máxima separación y escalabilidad	Sobrecarga estructural para el alcance actual y tres integrantes	Desproporcionada

Decisión

Se adopta MVVM + Repository con una capa de dominio ligera, organizada por funcionalidad.

Para la funcionalidad vertical Visualizar instalaciones deportivas se aplican las siguientes reglas:

1. La vista no contiene llamadas directas a Supabase.
2. El ViewModel depende únicamente de `InstitucionesRepository`, definido por el dominio.
3. El DTO `InstitucionDeportivaDto` y la entidad `InstitucionDeportiva` son clases diferentes.
4. El paso de DTO a entidad se realiza mediante `InstitucionDeportivaMapper`.
5. La implementación concreta del repositorio y el datasource pertenecen a la capa `data`.
6. El ensamblado de dependencias se realiza en `core/di/home\_dependencies.dart`.
7. La lógica de estado de interfaz utiliza `Loading`, `Success`, `Empty` y `ErrorState`.

Consecuencias

Positivas

- El `HomeViewModel` puede probarse sin emulador y sin Supabase real.
- Cambiar Supabase por otro backend afecta principalmente la capa `data`.
- La interfaz de dominio estabiliza el contrato que consume la presentación.
- Los estados de interfaz se vuelven explícitos y verificables.

Negativas

- Aumenta el número de archivos de la funcionalidad.
- El equipo debe revisar imports y evitar saltarse el repositorio.
- Requiere mantener el mapeo DTO ↔ entidad cuando cambie el backend.

Costo de revertir

Medio durante los primeros sprints y alto a partir del sprint 3, debido a que nuevas funcionalidades tenderán a reutilizar este patrón.

ADR-002 · Stack tecnológico de GameOn Network

- Estado: aceptada
- Fecha: 2026-09-04
- Decidido por: GameOn Team

Contexto

El equipo debe seleccionar un stack móvil mediante criterios ponderados y evidencia. GameOn Network ya dispone de una aplicación funcional en Flutter y requiere geolocalización, Google Maps, cámara, notificaciones, almacenamiento, reservas y conexión a servicios remotos.

Evaluación

El alcance actual del producto en el curso es Android. El criterio «Viabilidad sin macOS» se mantiene porque forma parte de la matriz obligatoria de la guía; en este proyecto no representa una necesidad de desarrollar para iOS.

La matriz completa se encuentra en `evaluacion_stacks.csv`. Con una escala de 1 a 5, los resultados ponderados son:

Stack	Puntaje
Flutter	4.90
Nativo	3.85
React Native	3.75
Kotlin Multiplatform	3.25

Dos finalistas para la prueba de humo

1. **Flutter** — stack del proyecto actual.
2. **React Native** — alternativa multiplataforma madura y razonable para comparación.

Capacidad crítica seleccionada

Mapa y geolocalización. Es una capacidad central de GameOn porque permite visualizar instalaciones deportivas cercanas y su ubicación.

Resultado de la prueba de humo

La ejecución real debe registrarse en `docs/evidencias/S02/` con captura o video de ambos stacks. El proyecto Flutter ya contiene integración con `google_maps_flutter` y `geolocator`; la evidencia de la ejecución debe obtenerse en el laboratorio. React Native deberá ejecutar un prototipo mínimo equivalente durante la prueba de 30 minutos.

No se declara una prueba de humo como aprobada sin captura o video real.

Decisión

Se continúa con Flutter porque:

- GameOn ya posee una base funcional implementada en este stack.
- El equipo cuenta con experiencia directa sobre el proyecto.
- Las capacidades críticas ya tienen dependencias integradas.
- Cambiar de stack provocaría una reimplementación extensa sin aportar valor proporcional al curso.

Plan de salida

Si Flutter dejara de ser viable para el alcance Android del proyecto, se considerará una migración hacia Kotlin/Android nativo o React Native cuando ocurra alguno de estos escenarios:

- una dependencia crítica deje de mantenerse y no exista reemplazo razonable;

- aparezcan restricciones de rendimiento demostrables mediante mediciones;
- Android requiera una capacidad crítica que no pueda resolverse de forma estable en Flutter;
- el equipo responsable del mantenimiento cambie y tenga competencias significativamente mayores en otro stack.

Qué puede reutilizarse

- Requerimientos, historias de usuario y criterios de aceptación.
- Diseño visual y flujos de navegación.
- Contratos de API y esquema de datos.
- Backend Supabase/Firebase.
- Casos de prueba, reglas de negocio y documentación ADR.

Qué debe reescribirse

- Vistas y navegación.
- ViewModels dependientes de Dart.
- Integraciones con paquetes Flutter.
- Configuración de compilación y CI específica del stack.

El costo de salida se considera alto, dado que la mayor parte de la interfaz móvil está implementada en Flutter.